Licenciatura en Sistemas de Información

Programación Avanzada - 2026

PROGRAMACIÓN AVANZADA · PRÁCTICA DE LABORATORIO

CI/CD con Firebase Hosting y hardening de headers HTTP

Preview channels por Pull Request, despliegue a producción, rollback y seguridad del lado del servidor de contenido estático.

Dato	Detalle
Duración	60 minutos en clase + ~20 minutos de preparación previa (obligatoria)
Modalidad	Individual o en parejas (un repo y un proyecto Firebase por equipo)
Stack	Next.js 16 (export estático) · GitHub Actions · Firebase Hosting
Costo	Cero. Plan Firebase **Spark**: no requiere tarjeta de crédito
Material	Repo base `practica-firebase-cicd` (provisto por la cátedra)
Ejes	Despliegue continuo · Infraestructura de entrega · Seguridad web

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

1. Contexto

Un equipo de desarrollo mantiene un **catálogo de cursos** hecho con Next.js. Hoy lo publican a mano con `firebase deploy` desde la computadora de alguien: no hay revisión previa en un entorno real, nadie sabe con certeza qué versión está en producción, y una auditoría de seguridad marcó que el sitio **no envía ningún header de seguridad**.

Tu trabajo no es programar features: la app ya funciona. Tu trabajo es construir **el camino a producción** y **robustecer lo que se sirve**:

- Que cada Pull Request se despliegue automáticamente en una **URL de preview** aislada, para revisarlo antes de mergear.
- Que lo que llega a `main` se publique en producción sin intervención manual, y que se pueda **volver atrás** en segundos.
- Que el sitio responda con los **headers HTTP de seguridad** adecuados y una **estrategia de caché** coherente.

2. Objetivos de aprendizaje

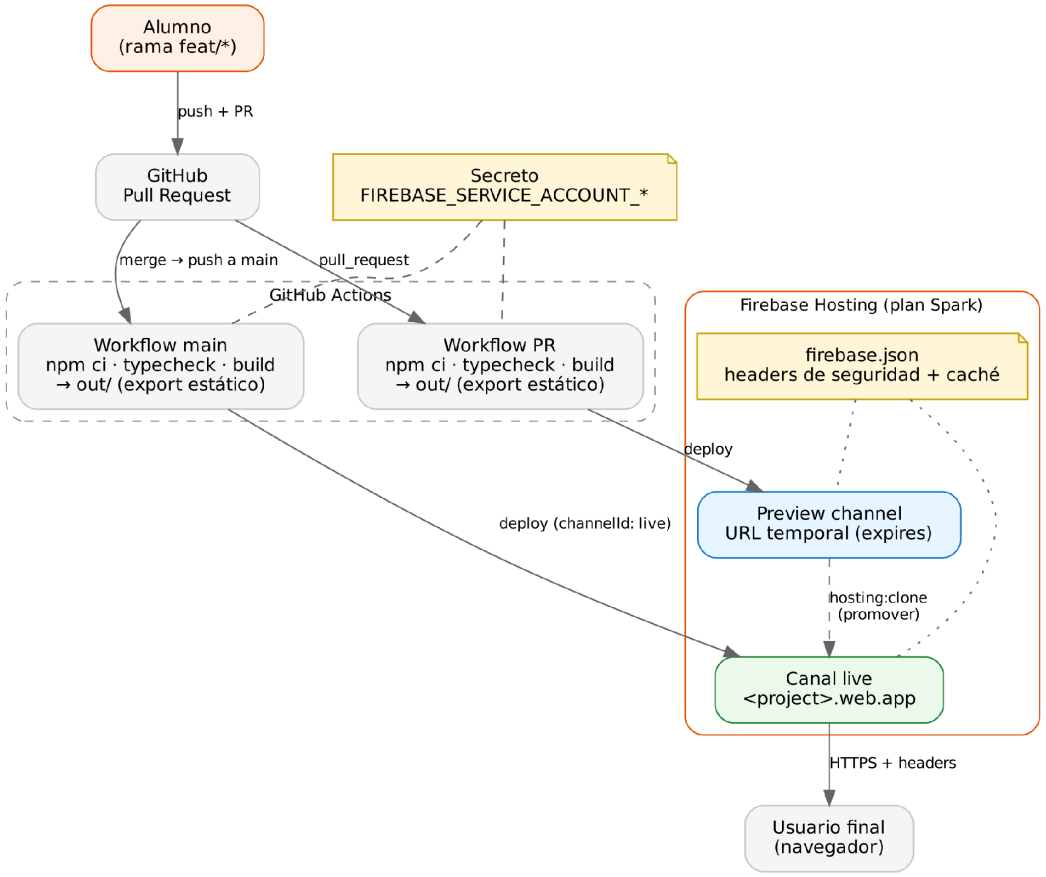
Al terminar la práctica deberías poder:

1. Diseñar un pipeline de CI/CD con **entornos efímeros** (preview por PR) y un entorno de **producción** separado.
2. Gestionar credenciales de despliegue como **secretos** del pipeline y razonar sobre sus permisos.
3. Garantizar **trazabilidad**: saber qué commit está desplegado en cada entorno.
4. Ejecutar un **rollback** y distinguir *desplegar* de *liberar*.
5. Configurar y justificar headers de seguridad (**CSP**, HSTS, X-Frame-Options, entre otros) y diagnosticar cuando rompen la aplicación.
6. Diseñar una **estrategia de caché** distinta para HTML y para assets versionados.

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

3. Arquitectura



Componentes y responsabilidades:

Componente	Responsabilidad	Dónde está
Next.js con `output: "export"`	`next build` genera HTML, JS y CSS estáticos en `out/`. No hay servidor.	`next.config.ts`
Workflow de PR	En cada Pull Request: instala, chequea tipos, compila y despliega a un **preview channel**.	`.github/workflows/firebase-hosting-pull-request.yml`
Workflow de producción	En cada push a `main`: mismo build, despliega al canal **live**.	`.github/workflows/firebase-hosting-merge.yml`
Service account	Identidad con permiso para desplegar. Su clave JSON vive como **secreto** de GitHub.	GitHub ⟶ Settings ⟶ Secrets
Firebase Hosting	CDN global con TLS. Canales: `live` (producción) y previews con vencimiento.	Consola de Firebase

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

Componente	Responsabilidad	Dónde está
`firebase.json`	Qué carpeta se publica, URLs limpias y **headers HTTP** de cada respuesta.	Raíz del repo
Footer de la app	Muestra commit, rama y fecha de build: así sabés qué versión ves.	`app/layout.tsx`

> **Decisión de diseño.** Firebase Hosting en plan Spark sirve solo archivos estáticos. Si la app necesitara SSR, Server Actions o rutas dinámicas, habría que usar App Hosting, que corre sobre Cloud Run y exige el plan Blaze (con cuenta de facturación). Elegir export estático es lo que mantiene el costo en cero.

4. Preparación previa (obligatoria, antes de la clase)

Sin esto la práctica no entra en 60 minutos. Si algo falla, avisá a la cátedra **antes** de la clase.

1. Tener **Node.js 20 o superior** y **Git** instalados, y una cuenta de GitHub.
2. Crear un proyecto en la consola de Firebase (plan Spark, sin Google Analytics) y anotar el **Project ID**.
3. Crear **tu propio repo** en GitHub a partir del repo base con **Use this template → Create a new repository**, en tu cuenta y como **público**. No trabajes sobre un fork: GitHub no expone secretos a PRs que vienen de forks.
4. Clonar **tu** repo (no el de la cátedra) con `git clone https://github.com/<tu-usuario>/practica-firebase-cicd.git`, ejecutar `npm install` y `npx firebase login`.
5. Reemplazar `REEMPLAZAR-POR-TU-PROJECT-ID` en `.firebaserc`.
6. Verificar en local: `npm run build` y `npm run serve`; la app tiene que abrir en `http://localhost:5000`.
7. Ejecutar `npx firebase init hosting:github` y elegir tu repo. Si pregunta por sobrescribir workflows existentes, responder **No**. Al terminar, confirmar en GitHub que existe un secreto `FIREBASE_SERVICE_ACCOUNT_...` y **anotar su nombre exacto**.
8. Hacer un primer deploy manual: `npx firebase deploy --only hosting` y abrir `https://<PROJECT_ID>.web.app`.
9. **Verificar que podés pushear cambios en workflows.** GitHub rechaza por HTTPS cualquier cambio en `.github/workflows/` si tu credencial no tiene el permiso `workflow`, y en la Parte A vas a editar esos archivos. Probalo con los comandos de abajo y, si funciona, borrá la rama de prueba.

```
git checkout -b chore/test-workflow
echo "# prueba de permisos" >> .github/workflows/firebase-hosting-merge.yml
git commit -am "chore: prueba de permisos de workflow"
git push -u origin chore/test-workflow
# si funcionó: limpiar
git checkout main && git branch -D chore/test-workflow
git push origin --delete chore/test-workflow
```

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

Si el push falla con un mensaje como *refusing to allow a Personal Access Token to create or update workflow … without workflow scope*, tu credencial no tiene ese permiso. Opciones:

• Con GitHub CLI: `gh auth login` y después `gh auth refresh -s workflow`.

• Con token personal: crear uno con el scope `workflow` (token clásico) o con *Workflows: Read and write* (token fine-grained) y usarlo como contraseña al pushear.

• Último recurso en clase: editar los YAML desde la web de GitHub (lápiz → *Commit changes* en una rama nueva → abrir PR).

Llegás a clase con: la app visible en `https://<PROJECT_ID>.web.app`, el nombre del secreto anotado, el repo clonado en tu máquina y la prueba de push de workflows superada

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

5. Cronograma de la clase

Minuto	Bloque	Actividad
00 – 05	Introducción	Recorrido por la arquitectura y el repo base.
05 – 10	Chequeo	Cada equipo confirma que la preparación previa funciona (app online, secreto creado).
10 – 25	Parte A	Completar los workflows (A1–A3), abrir el primer PR y obtener la URL de preview.
25 – 30	Parte A	Merge a `main`, verificar producción y practicar rollback (A6).
30 – 48	Parte B	Medición inicial, headers de seguridad, CSP y caché, probando en local con el emulador.
48 – 55	Parte B	Llevar los cambios por PR: validar en preview y publicar en producción (B5).
55 – 60	Cierre	Puesta en común de los problemas encontrados y entrega.

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

6. Parte A · Pipeline de CI/CD

Los dos workflows del repo están **incompletos a propósito**: mientras su trigger sea `workflow_dispatch`, no corren solos. Buscá los comentarios `TODO A?` en `.github/workflows/`.

A1 · Triggers

Configurá el workflow de preview para que se ejecute en cada **Pull Request**, y el de producción para que se ejecute en cada **push a `main`**.

Logro: al abrir un PR aparece una ejecución en la pestaña *Actions*.

A2 · Trazabilidad de la versión

Completá las variables `NEXT_PUBLIC_BUILD_SHA` y `NEXT_PUBLIC_BUILD_REF` en ambos workflows usando el contexto de GitHub Actions (las pistas están en los comentarios). Se inyectan en tiempo de build y se ven en el footer de la app.

Logro: el footer del preview muestra el SHA del último commit del PR y el nombre de la rama; el de producción muestra el SHA de `main`.

A3 · Credenciales y canales

- Usá el **nombre exacto** del secreto que creó `firebase init hosting:github` y tu **Project ID**.
- En el preview, definí un vencimiento con `expires` (por ejemplo, `3d`).
- En producción, definí el `channelId` que publica en el canal en vivo.

Logro: el bot comenta en el PR la URL del preview; al mergear, el cambio aparece en `https://<PROJECT_ID>.web.app`.

A4 · Concurrencia (opcional)

Agregá bloques `concurrency` con criterios distintos: en el preview, un push nuevo al mismo PR **cancela** el build anterior; en producción, los deploys **esperan en fila** y nunca se cortan a la mitad.

A5 · Environment de producción (opcional)

Asociá el job de producción a un GitHub Environment `production`. Explorá en *Settings* → *Environments* cómo exigir una aprobación manual antes del deploy.

A6 · Ciclo completo y rollback

1. Creá una rama `feat/titulo`, cambiá el título de `app/page.tsx`, pusheá y abrí un PR.
2. Verificá el cambio en la URL de preview y comprobá que **producción no cambió**.
3. Mergeá el PR y verificá que producción muestre el nuevo título y el nuevo SHA.
4. Hacé rollback desde la consola: *Hosting* → *Release history* → ⋮ → *Rollback*. Verificá en el footer que volvió el SHA anterior.
5. Bonus: promové un preview a producción sin recompilar con `npx firebase hosting:clone <PROJECT_ID>:<CANAL> <PROJECT_ID>:live`.

> **Pregunta para pensar:** después del rollback, `main` sigue teniendo el código nuevo. ¿Qué pasa con el próximo merge? ¿Cómo se resuelve en un equipo real?

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

7. Parte B · Hardening de headers HTTP

Todo se configura en la clave `hosting.headers` de `firebase.json`; no hay código de backend. El formato es:

```
"headers": [
  {
    "source": "**",
    "headers": [
      { "key": "X-Content-Type-Options", "value": "nosniff" }
    ]
  }
]
```

> **Ciclo de prueba en local:** `npm run build` → `npm run serve` → `node scripts/check-headers.mjs http://localhost:5000`. El emulador aplica los headers de `firebase.json`; `npm run dev` **no** los aplica.

B1 · Medición inicial

Medí producción **antes** de tocar nada, y guardá la salida:

```
node scripts/check-headers.mjs https://<PROJECT_ID>.web.app
```

Medí también con securityheaders.com. Anotá la calificación inicial.

B2 · Headers básicos

Agregá, para todas las rutas, los siguientes headers. Para cada uno vas a tener que justificar en la reflexión **qué ataque mitiga**:

Header	Pista
`Strict-Transport-Security`	Obliga a usar HTTPS en visitas futuras. Pensá el `max-age`.
`X-Content-Type-Options`	Evita que el navegador "adivine" el tipo de un archivo.
`X-Frame-Options`	¿Quién puede embeber tu sitio en un iframe?
`Referrer-Policy`	Qué parte de tu URL se envía a otros sitios al navegar.
`Permissions-Policy`	Qué APIs del navegador (cámara, micrófono, ubicación) puede usar la página.

B3 · Content-Security-Policy

Definí una CSP que solo permita recursos del propio sitio. Punto de partida sugerido:

```
default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self';
object-src 'none'; base-uri 'self'; frame-ancestors 'none'
```

Aplicala, reconstruí y abrí `http://localhost:5000/cursos` con DevTools abierto. **Algo se va a romper.** Esa es la parte central de la práctica:

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

- Leé los errores de la consola: indican qué directiva bloqueó qué recurso.
- Investigá **por qué** pasa. Pista: mirá el código fuente del HTML generado en `out/cursos.html`.
- Corregí la política para que la app vuelva a funcionar con el **menor permiso posible**, y documentá qué decidiste y qué costo de seguridad tiene.
- No te olvides de la imagen de portada: ¿de qué orígenes viene realmente? Mirá la pestaña *Network*.

B4 · Estrategia de caché

Configurá `Cache-Control` para que:

- Los archivos de `/_next/static/**` (nombres con hash de contenido) se cacheen por **un año** como inmutables.
- El HTML **no** quede cacheado sin revalidar (si no, un deploy o un rollback tardaría en verse).

> **Dato útil:** si dos reglas que matchean la misma URL definen el mismo header, gana **la última** del arreglo. Aprovechalo para la regla general y la específica.

B5 · Validar en preview y publicar

1. Llevá los cambios por PR (no directo a `main`).
2. Ejecutá `check-headers.mjs` contra la **URL del preview** y verificá que la app funciona ahí.
3. Mergeá y repetí la medición en producción. Compará con la medición inicial.

Logro: `check-headers.mjs` termina con "Todos los controles pasaron" en producción, y el filtro de `/cursos` y la imagen de portada funcionan.

Desafío extra · CSP sin 'unsafe-inline'

Si para que la app funcione tuviste que permitir `'unsafe-inline'` en `script-src`, intentá eliminarlo. Como no hay servidor, no se puede usar un *nonce* por request. Explorá la alternativa de **hashes** (`'sha256-...'`): ¿cómo los calcularías y en qué paso del pipeline? Puntúa como destacado.

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

8. Preguntas de reflexión

Respondelas en un archivo `REFLEXION.md` en la raíz de tu repo (respuestas breves y concretas):

1. ¿Por qué el preview tiene vencimiento? ¿Qué riesgos tendría un preview que no vence nunca?
2. ¿Qué diferencia hay entre *desplegar* y *liberar*? ¿Dónde aparece cada concepto en este flujo?
3. ¿Por qué la clave de la service account va en un secreto y no en el repo? ¿Qué podría hacer alguien que la obtenga? ¿Cómo limitarías el daño?
4. La CSP rompió la app: ¿por qué? ¿Qué opciones evaluaste y qué costo de seguridad tiene cada una?
5. Para cada header de B2, ¿qué ataque mitiga?
6. ¿Por qué los assets de `/_next/static/` pueden cachearse un año y el HTML no?
7. Si la app necesitara SSR, ¿qué cambiaría en la arquitectura, en la CSP y en el costo?

9. Entregables

#	Entregable	Formato
1	Link al repo (público, o privado con la cátedra como colaboradora)	URL
2	URL de producción `https://<PROJECT_ID>.web.app`	URL
3	Link a un PR mergeado con el comentario del bot y la URL de preview	URL
4	Salida de `check-headers.mjs` **antes** (B1) y **después** (B5)	Texto o captura en `REFLEXION.md`
5	Respuestas a las preguntas de reflexión	`REFLEXION.md`

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

10. Rúbrica de evaluación

Criterio	Pts	Insuficiente	Esperado	Destacado
Pipeline de preview	20	No corre o no despliega.	Cada PR despliega a un preview con vencimiento y el bot comenta la URL.	Además, `concurrency` con cancelación y gate de verificación sobre el preview.
Pipeline de producción	15	Deploy manual o al canal equivocado.	El push a `main` publica en `live`.	Además, `concurrency` sin cancelación y environment `production`.
Trazabilidad y rollback	15	No se sabe qué versión está desplegada.	El footer muestra SHA y rama correctos; rollback ejecutado y verificado.	Promoción de preview a live con `hosting:clone` y análisis del estado de `main` tras el rollback.
Headers de seguridad	20	Faltan headers o están mal escritos.	Los cinco headers de B2 presentes y justificados.	Valores ajustados con criterio (p. ej. COOP, `Permissions-Policy` completa).
Content-Security-Policy	15	Sin CSP, o la app queda rota.	CSP funcional, con `'unsafe-inline'` justificado si se usó; imagen permitida por origen exacto.	CSP sin `'unsafe-inline'` (hashes calculados en el pipeline).
Estrategia de caché	10	Sin `Cache-Control` o igual para todo.	Assets inmutables por un año; HTML con revalidación.	Explica el impacto en deploys y rollbacks.
Reflexión	5	Incompleta o genérica.	Responde todas con fundamentos.	Relaciona con casos reales o alternativas de arquitectura.

Total: 100 puntos. Se aprueba con 60, siempre que los criterios de pipeline de preview y headers de seguridad estén al menos en nivel *Esperado*.

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

Anexo A · Comandos útiles

Comando	Para qué
`npm run dev`	Desarrollo con hot reload (no aplica los headers de `firebase.json`).
`npm run build`	Export estático a `out/`.
`npm run serve`	Emulador de Hosting en `localhost:5000`, **con** headers.
`npm run typecheck`	Chequeo de tipos (también corre en el pipeline).
`node scripts/check-headers.mjs <URL>`	Verifica headers de seguridad y caché; sale con código 1 si falta algo.
`curl -I <URL>`	Ver los headers crudos de una respuesta.
`npx firebase hosting:channel:list`	Lista los previews y sus vencimientos.
`npx firebase hosting:channel:delete <canal>`	Borra un preview.
`npx firebase hosting:clone <P>:<canal> <P>:live`	Promueve un preview a producción sin recompilar.

Anexo B · Problemas frecuentes

Síntoma	Causa probable
El workflow no aparece en *Actions*	El trigger sigue en `workflow_dispatch` (A1), o el archivo no está en la rama del PR.
`Input required and not supplied: firebaseServiceAccount`	Nombre del secreto distinto, o el PR viene de un fork.
Footer muestra `TODO` o `local`	Faltan las variables `NEXT_PUBLIC_BUILD_*` en el paso de build (A2).
`/cursos` da 404	`cleanUrls` no está en `true`: el export genera `cursos.html`.
La página carga pero el filtro no responde	La CSP bloquea scripts. Revisá la consola de DevTools.
La imagen de portada no aparece	`img-src` no incluye todos los orígenes involucrados (mirá *Network*).
Cambié `firebase.json` y no veo cambios	Reiniciá el emulador; en el navegador probá ventana privada o `curl -I`.

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

Síntoma	Causa probable
Push rechazado: *refusing to allow … without workflow scope*	La credencial HTTPS no tiene permiso `workflow`. `gh auth refresh -s workflow`, token con scope `workflow`, o editar el YAML desde la web de GitHub.
Push rechazado con 403 / *Permission denied* al repo de la cátedra	Clonaste el repo de la cátedra en lugar del tuyo. Creá tu repo con *Use this template* y apuntá el remoto: `git remote set-url origin https://github.com/<tu-usuario>/practica-firebase-cicd.git`.

Anexo C · Referencias

- Firebase Hosting — Integración con GitHub: firebase.google.com/docs/hosting/github-integration
- Firebase Hosting — Configuración (`firebase.json`): firebase.google.com/docs/hosting/full-config
- Action oficial: github.com/FirebaseExtended/action-hosting-deploy
- Next.js — Static Exports: nextjs.org/docs/app/guides/static-exports
- MDN — Content Security Policy: developer.mozilla.org/docs/Web/HTTP/CSP
- OWASP — HTTP Headers Cheat Sheet: cheatsheetseries.owasp.org